

Processing.js:

Processing.js is a JavaScript-based adaptation of the original Processing language, designed to bring Processing sketches from Java into the web browser. It allows developers to create graphics, animations, and interactive visuals using HTML5 canvas with a syntax similar to Java, often written in `.pde` style. Processing.js played an important role in making creative coding accessible on the web, but it is now considered outdated due to limited modern support and the rise of more advanced libraries.

p5.js:

p5.js is a modern JavaScript library inspired by Processing that focuses on making creative coding simple, interactive, and accessible for beginners and developers alike. It uses standard JavaScript syntax and integrates easily with web technologies like HTML and CSS, allowing users to build animations, visualizations, and interactive applications directly in the browser. With strong community support, continuous updates, and features like sound, DOM manipulation, and interactivity, p5.js has become the preferred tool for creative coding on the web today.

Basic Drawing Functions

// functions in bold are most important!!!!

point(x, y);

line(x1, y1, x2, y2);

rect(x, y, width, height);

rect(x, y, width, height, radius); // rounded

ellipse(x, y, width, height);

arc(x, y, width, height, start, stop);

triangle(x1, y1, x2, y2, x3, y3);

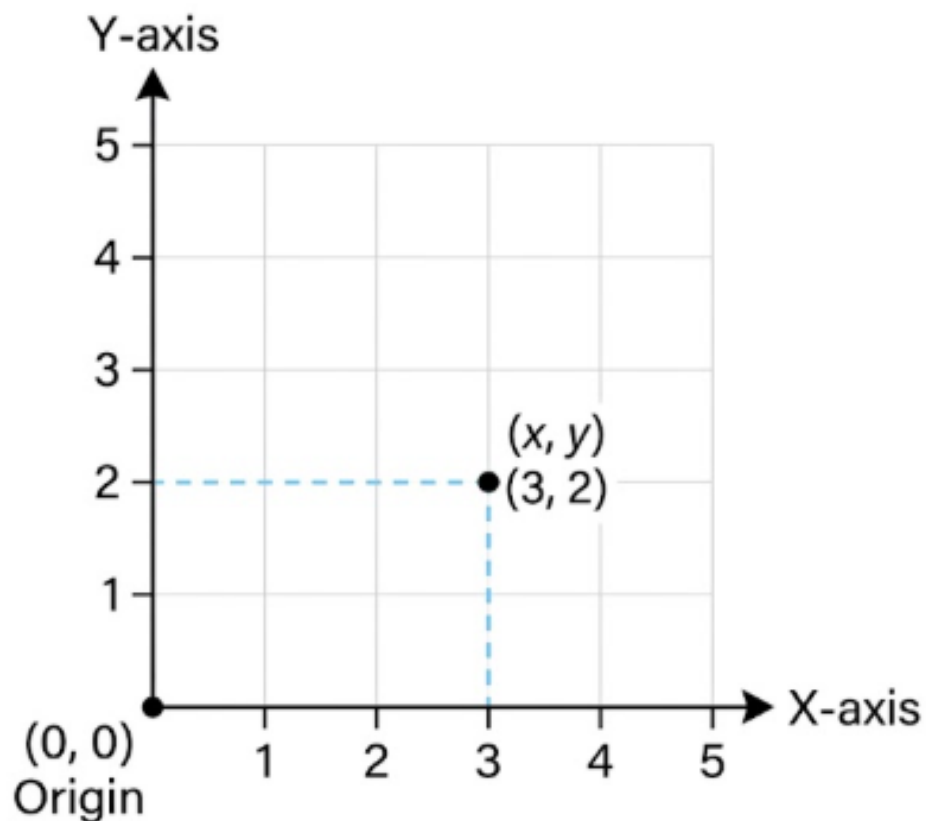
quad(x1, y1, x2, y2, x3, y3, x4, y4);

```
// Custom shape (polygon)
beginShape();
vertex(x1, y1);
vertex(x2, y2);
vertex(x3, y3);
...
endShape(CLOSE);

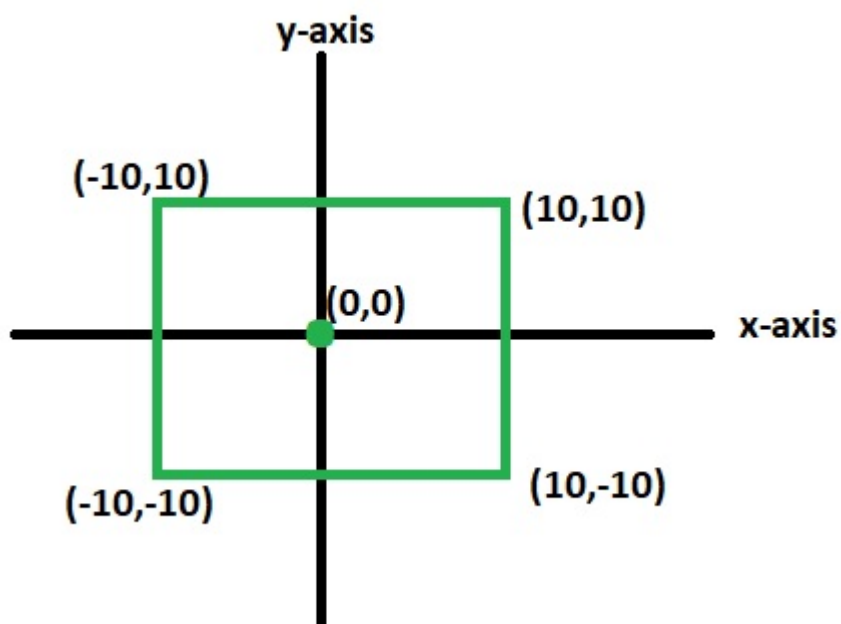
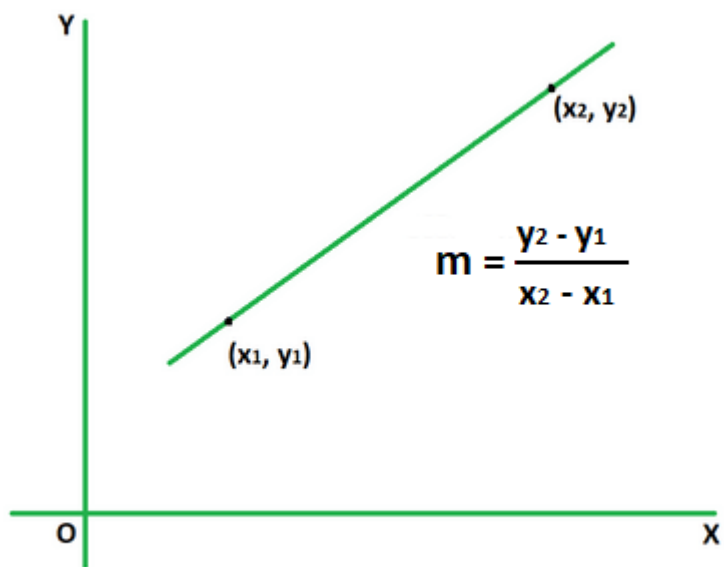
// Curves
bezier(x1, y1, cx1, cy1, cx2, cy2, x2, y2);

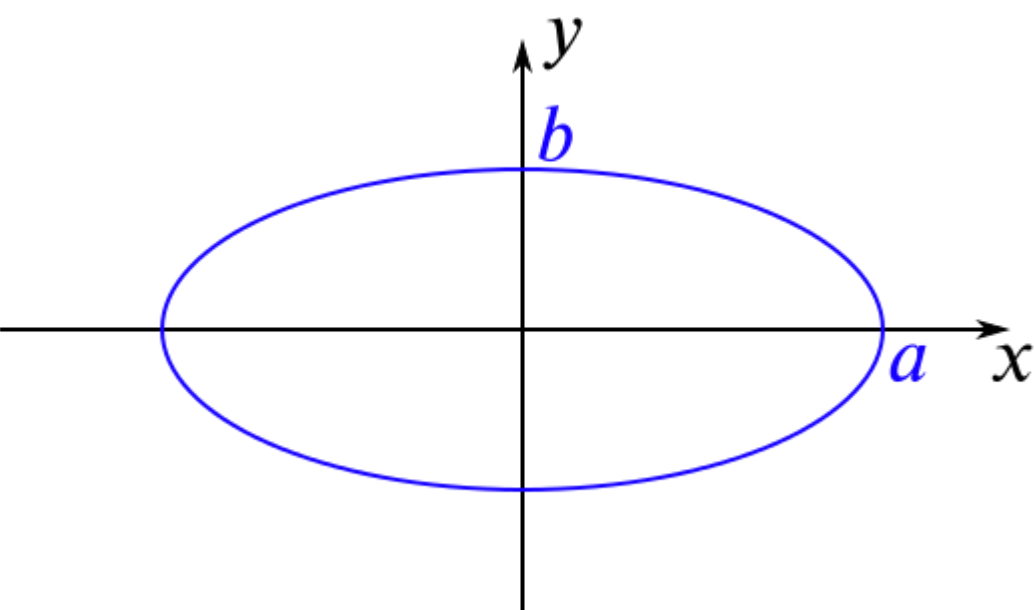
curve(x1, y1, x2, y2, x3, y3, x4, y4);
```

Diagrammatic Representation

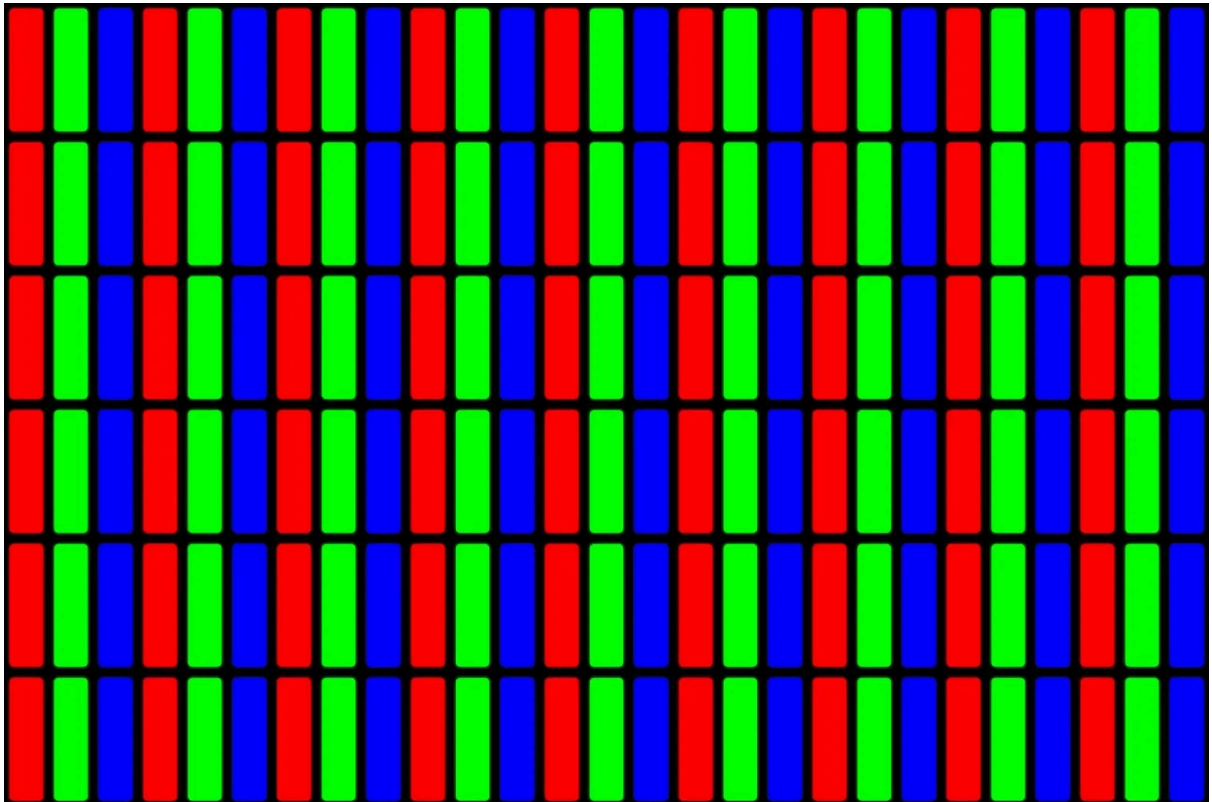
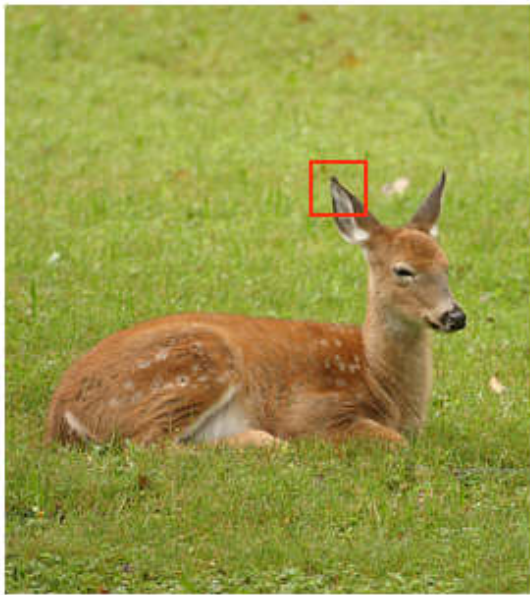


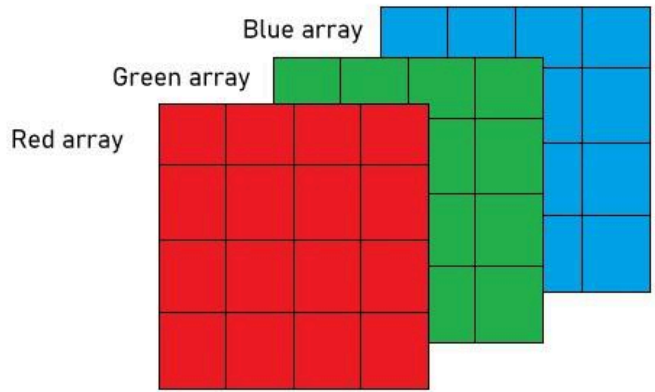
INTRO TO JS: DRAWING AND ANIMATION





RGB SYSTEM AND COLOURING





Arrays stacked over each other to form a Digital Image.

The **RGB color model** stands for **Red, Green, and Blue**, which are the three primary colors of light used to create different colors on digital screens. In this model, colors are formed by combining different intensities of red, green, and blue, where each value ranges from **0 to 255**.

- $(0, 0, 0) \rightarrow$ Black (no color)
- $(255, 255, 255) \rightarrow$ White (full intensity)
- $(255, 0, 0) \rightarrow$ Red
- $(0, 255, 0) \rightarrow$ Green
- $(0, 0, 255) \rightarrow$ Blue

By mixing these values in different proportions, millions of colors can be generated.

👉 RGB ka matlab hota hai **Red, Green aur Blue**

👉 Yeh teen colors milke screen par saare colors banate hain

- 0 matlab color off
- 255 matlab full color

👉 Simple bole toh:

“RGB values change karke hum koi bhi color bana sakte hain.”

COLOURING FUNCTIONS

🎨 **1. fill() → Shape ke andar ka color**

`fill(r, g, b);`

✅ **Example:**

```
fill(255, 0, 0); // red color
rect(100, 100, 100, 100);
```

🎨 **2. stroke() → Border (outline) color**

`stroke(r, g, b);`

✅ **Example:**

```
stroke(0, 0, 255); // blue border
line(50, 50, 200, 200);
```

🎨 **3. background() → Background color**

`background(r, g, b);`

✅ **Example:**

```
background(200, 200, 200);
```

🎨 **4. noFill() → No inside color**

`noFill();`

5. noStroke() → No border

noStroke();

6. strokeWeight() → Border thickness

strokeWeight(value);

- 👉 `fill()` → shape ke andar ka color set karta hai
- 👉 `stroke()` → shape ka border color set karta hai
- 👉 `background()` → poore canvas ka background color set karta hai
- 👉 `noFill()` → andar ka color hata deta hai
- 👉 `noStroke()` → border hata deta hai
- 👉 `strokeWeight()` → border ki thickness badhata hai

CODE EXAMPLE:

```
void setup() {  
  size(400, 400);  
}  
  
void draw() {  
  background(255);  
  
  fill(255, 0, 0);  
  stroke(0);  
  strokeWeight(3);  
  rect(100, 100, 200, 150);  
}
```